

## Módulo 6 – Capítulo 01 – Sección 01

---

### Qué es RAG y por qué existe: limitaciones de los LLM sin contexto externo

En el Módulo 5 aprendiste a construir aplicaciones con LLMs: a consumir la API de Claude o GPT-4o, a diseñar prompts efectivos, a encadenar llamadas con LangChain y a construir aplicaciones conversacionales básicas. Con esas herramientas puedes construir sistemas que razonan, resumen, clasifican y generan texto con notable calidad. Pero hay una clase de problemas que todas esas técnicas no pueden resolver: ¿qué ocurre cuando el usuario pregunta sobre un contrato que tu empresa firmó hace tres meses? ¿O sobre la política de vacaciones actualizada en enero? ¿O sobre el estado de un incidente de soporte que ocurrió ayer? El LLM que integraste en el Módulo 5 no sabe nada de eso, y no puede saberlo: fue entrenado sobre datos anteriores a una fecha de corte, y sus pesos no cambian en tiempo de inferencia. Retrieval-Augmented Generation (RAG) es la arquitectura que resuelve exactamente este problema.

Retrieval-Augmented Generation, introducido por Lewis et al. en el paper "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" (2020), convierte cada inferencia en una consulta activa a una base de conocimiento externa. En lugar de depender exclusivamente de lo que el modelo "recuerda" de su entrenamiento, el sistema RAG recupera los fragmentos de documentos más relevantes para la pregunta del usuario e inyecta esa información en el contexto del prompt antes de que el LLM genere su respuesta. El resultado es un sistema que combina la capacidad de razonamiento lingüístico del LLM con acceso dinámico a fuentes verificables y actualizables: el LLM aporta la comprensión del lenguaje y la síntesis; el índice externo aporta el conocimiento factual específico del dominio.

La necesidad de RAG emerge de una limitación estructural de los LLMs que es importante comprender en profundidad: la separación entre conocimiento paramétrico y conocimiento no paramétrico. El conocimiento paramétrico es el que el modelo internalizó durante el entrenamiento y reside en sus miles de millones de parámetros; puede activarse sin contexto externo pero está congelado en la fecha de corte del entrenamiento. El conocimiento no paramétrico es el que se inyecta en el contexto de cada inferencia mediante el prompt; puede

actualizarse en tiempo real pero requiere un mecanismo de recuperación que identifique qué información es relevante para cada consulta específica. RAG es, en esencia, el sistema de recuperación que convierte el conocimiento no paramétrico en algo escalable a millones de documentos y miles de consultas por día.

Para entender por qué RAG no es solo una extensión del prompting directo que practicaste en el Módulo 5, considera la aritmética del problema: un corpus empresarial típico tiene entre 10.000 y 10 millones de documentos, y cada documento puede tener decenas de páginas. Incluir todo ese corpus en el contexto de cada inferencia es técnicamente imposible: incluso los modelos con ventanas de 1 millón de tokens (Gemini 1.5 Pro) no pueden contener más de unos cientos de documentos completos, y el costo por inferencia escalaría linealmente con el volumen de contexto. RAG resuelve este problema con una búsqueda de vecinos más cercanos en el espacio vectorial del corpus, recuperando solo los 5-20 fragmentos más relevantes para cada consulta, con latencias de milisegundos y costo independiente del tamaño total del corpus.

## Limitaciones fundamentales de los LLM sin recuperación externa

- **Knowledge cutoff:** los pesos del modelo encodifican conocimiento hasta la fecha de entrenamiento; GPT-4o fue entrenado con datos hasta abril de 2024, Claude 3.5 Sonnet hasta marzo de 2025; cualquier evento, documento o actualización posterior a esa fecha es invisible para el modelo sin RAG.
- **Context window como único canal de conocimiento en inferencia:** sin RAG, el único modo de inyectar información específica es incluirla manualmente en el prompt, lo que no escala a corpora de millones de documentos ni a organizaciones con miles de documentos privados.
- **Hallucination de hechos específicos:** el modelo interpola entre patrones estadísticos y puede fabricar nombres, fechas, cifras o citas con alta fluidez pero baja fidelidad cuando no tiene evidencia en sus pesos; en un experimento clásico, GPT-3.5 producía citas bibliográficas plausibles pero inexistentes para preguntas académicas que desconocía.
- **Opacidad de fuentes:** un LLM sin RAG no puede citar la fuente de una afirmación porque no la recuperó; RAG hace el conocimiento trazable y auditable por diseño, permitiendo al usuario verificar cada fragmento de información en el documento original.
- **Costo prohibitivo de actualización por fine-tuning continuo:** reentrenar o hacer fine-tuning del modelo para incorporar nuevos documentos tiene un costo computacional

de decenas o cientos de miles de dólares y no está al alcance de la mayoría de los equipos; RAG externaliza el conocimiento al índice y permite actualizaciones en segundos.

**Nota del Arquitecto:** *En producción, la primera pregunta que debes responder antes de diseñar un sistema RAG es: "¿Con qué frecuencia cambia el corpus y cuántos documentos privados tiene la organización?" Si el corpus tiene menos de 50 documentos que raramente cambian, el context stuffing directo puede ser más simple y efectivo que RAG. Si el corpus tiene miles de documentos o cambia diariamente, RAG es el único enfoque operativamente viable. Esta decisión se profundiza en la Sección 5 de este capítulo.*

La comprensión de estas limitaciones no es académica: son las mismas razones por las que Notion AI, GitHub Copilot y los sistemas de asistencia interna de empresas como Anthropic, Databricks y Elastic usan RAG como arquitectura central. El conocimiento que esos sistemas necesitan está distribuido en millones de documentos propietarios que no pueden incluirse en un prompt y que se actualizan constantemente. RAG es la única arquitectura que resuelve este problema a escala operativa.

Con el "por qué" establecido, la siguiente sección disecciona el "cómo": la arquitectura técnica de un sistema RAG canónico, con sus cuatro etapas y los componentes específicos que implementan cada una de ellas.

## Módulo 6 – Capítulo 01 – Sección 02

---

### Arquitectura básica de RAG: ingesta, indexación, recuperación y generación

Si en la sección anterior respondimos por qué RAG existe, en esta respondemos cómo funciona: cuáles son sus cuatro etapas, qué componentes implementan cada una, y por qué la separación entre el plano offline y el plano online es la decisión arquitectónica que hace al sistema escalable. Comprender esta arquitectura canónica es el punto de partida obligatorio antes de explorar las variantes y optimizaciones que cubren los capítulos siguientes.

Un sistema RAG canónico se descompone en cuatro etapas que operan sobre planos de tiempo distintos. Las dos primeras —ingesta y indexación— son procesos offline o near-realtime que preparan el corpus; las dos últimas —recuperación y generación— son operaciones online que responden cada consulta del usuario en tiempo de inferencia. Esta separación temporal es fundamental: el índice vectorial se construye una vez (o se actualiza incrementalmente) y puede ser consultado millones de veces sin reconstruirse. Un corpus de 100.000 documentos que tardó 6 horas en indexarse puede responder 10.000 consultas simultáneas en milisegundos porque la búsqueda ANN opera sobre el índice ya construido, no sobre los documentos originales.

La etapa de **ingesta** transforma documentos crudos —PDFs corporativos, páginas HTML, registros de bases de datos, exportaciones de Confluence o Notion— en chunks de texto mediante loaders y splitters especializados. LangChain ofrece más de 100 document loaders que manejan formatos específicos: `PyMuPDFLoader` para PDFs con soporte de extracción de texto por página, `BeautifulSoupTransformer` para contenido HTML, `CSVLoader` para datos tabulares. La etapa de **indexación** convierte esos chunks en vectores de alta dimensionalidad mediante un modelo de embedding —text-embedding-3-small de OpenAI, voyage-3 de Voyage AI, o BGE-M3 en hosting propio— y persiste el par (vector, metadatos) en una base de datos vectorial como Qdrant, Pinecone o pgvector. El vector captura la semántica del chunk de forma que los vectores de contenidos similares sean cercanos en el espacio geométrico, habilitando la búsqueda por similitud.

En tiempo de consulta, el **retriever** convierte la query del usuario en un vector de embedding usando el mismo modelo que se usó en la indexación —condición no negociable: cambiar el modelo de embedding requiere reindexar todo el corpus— y ejecuta una búsqueda ANN (Approximate Nearest Neighbor) en el índice, devolviendo los K chunks con mayor similitud coseno al vector de la query. El número K es configurable: valores entre 3 y 10 son los más comunes en producción. Finalmente, el **generador** recibe una solicitud que combina la query original con los K chunks recuperados en un prompt estructurado, y produce la respuesta fundamentada en la evidencia recuperada. El prompt template define exactamente cómo se inyecta el contexto: típicamente algo como "Basándote en los siguientes documentos, responde la pregunta. Si la respuesta no está en los documentos, indica que no tienes información suficiente. Documentos: {context}. Pregunta: {question}".

## Componentes principales de la arquitectura RAG

- **Document loaders:** parsers específicos por tipo de fuente —PyMuPDF para PDFs, BeautifulSoup para HTML, conectores de bases de datos— que extraen texto plano y metadatos estructurados (nombre del archivo, número de página, fecha de creación, URL origen).
- **Text splitters:** algoritmos de chunking que segmentan el texto en unidades indexables; `RecursiveCharacterTextSplitter` de LangChain es el estándar de facto para texto libre no estructurado, aplicando separadores jerárquicos ("`\n\n`", "`\n`", "`.`", "`\"`") para dividir en límites semánticamente significativos.
- **Embedding models:** modelos como text-embedding-3-small (OpenAI, \$0.02/M tokens) o voyage-3 (Voyage AI, \$0.006/M tokens) que proyectan chunks de texto a vectores densos en espacios de 256 a 3072 dimensiones.
- **Vector store:** base de datos especializada que persiste los vectores y sus metadatos, y expone una interfaz de búsqueda por similitud coseno o producto punto con filtrado por atributos de metadatos.
- **Retriever:** componente que ejecuta la query de búsqueda vectorial, aplicando opcionalmente filtros de metadatos (solo documentos posteriores a 2024-01-01, solo categoría "legal") y reranking, retornando los top-K chunks con sus scores de similitud.
- **Prompt template:** plantilla que estructura la inyección del contexto recuperado junto a la pregunta del usuario, definiendo el formato e instrucciones que recibe el LLM generador para responder con fidelidad al contexto.

**Nota del Arquitecto:** *El error más común en implementaciones iniciales de RAG es usar el mismo modelo de embedding en desarrollo y cambiarlo en producción porque "el otro tiene mejor rendimiento en MTEB". Este cambio requiere reindexar todo el corpus, porque los vectores generados por modelos distintos son incomparables incluso si tienen la misma dimensionalidad. Elige el modelo de embedding con cuidado desde el inicio y trátalo como una dependencia versionada del sistema.*

La separación entre el plano offline (ingesta + indexación) y el plano online (recuperación + generación) no es solo una decisión de rendimiento: es lo que hace posible que el corpus crezca de 1.000 a 10 millones de documentos sin modificar una sola línea del pipeline de serving. El retriever siempre consulta el índice construido; el tamaño de ese índice impacta el costo de almacenamiento y la latencia de búsqueda ANN, pero con algoritmos como HNSW (que veremos en el Capítulo 3) ese impacto es logarítmico en N, no lineal.

La siguiente sección describe el ciclo de vida completo de un documento dentro de este sistema: desde que ingresa como archivo binario hasta que su vector está disponible para consulta, con todos los pasos de transformación que ocurren en el camino y los puntos donde un error puede propagar degradación a toda la cadena.

---

## Módulo 6 – Capítulo 01 – Sección 03

---

### El ciclo de vida de un documento en un sistema RAG

Cada documento que entra a un sistema RAG experimenta una transformación profunda: de ser un archivo binario opaco que solo un humano puede leer, se convierte en decenas de vectores matemáticos que un algoritmo de búsqueda puede comparar contra una query en microsegundos. Comprender este ciclo de vida en detalle es crucial porque los errores que ocurren en cualquiera de sus etapas no son corregibles después: un documento mal extraído produce chunks incorrectos, que generan vectores de baja calidad, que producen recuperaciones fallidas, que resultan en respuestas incorrectas del LLM. La calidad del índice es el techo de calidad de todo el sistema RAG, y ese techo se fija en el momento de la ingesta.

El ciclo comienza con la **extracción**: parsers especializados convierten el binario en texto plano preservando la estructura semántica del documento original. Un contrato legal en PDF tiene una jerarquía de cláusulas, artículos y subcláusulas que un simple `pdfplumber.extract_text()` puede destruir al mezclar columnas o ignorar los encabezados de sección. Herramientas como Unstructured.io, PyMuPDF y Docling de IBM invierten en detectar el layout del documento —columnas, tablas, figuras, encabezados— y reconstruyen el orden de lectura lógico que puede diferir del orden de bytes en el archivo. Docling es especialmente valioso para documentos técnicos con diagramas y ecuaciones: usa modelos de visión para detectar figuras y genera captions automáticos de ellas, habilitando su indexación como texto.

La etapa de **limpieza y normalización** elimina el ruido que degradaría la calidad de los embeddings: encabezados y pies de página que se repiten en cada página del PDF ("Confidencial — Uso Interno — Página N de M"), caracteres de control, espacios múltiples, texto de artefactos OCR en documentos escaneados. Adicionalmente, la detección de idioma (con herramientas como `langdetect` o `fasttext-language-identification`) permite enrutar el texto a modelos de embedding apropiados para el idioma del documento, crítico en corpus multilingües donde un modelo monolingüe produce embeddings de baja calidad para idiomas que no representó bien en su entrenamiento.

El **chunking** divide el texto limpio en fragmentos indexables con parámetros de tamaño y overlap configurables. Un chunk de 512 tokens con 10% de overlap (50 tokens compartidos con el chunk siguiente) produce aproximadamente 200 palabras de texto con redundancia suficiente para evitar que información que cae en el límite de un chunk sea inaccesible. El chunking no es una operación neutra: un chunk que divide una cláusula legal en el punto medio de la condición produce un fragmento sin sentido autónomo, con un embedding que no captura la semántica de la cláusula completa. El Capítulo 4 está dedicado enteramente a las estrategias de chunking y su impacto en la calidad de la recuperación.

El **enriquecimiento de metadatos** es la etapa que transforma un chunk de texto anónimo en un fragmento trazable: se adjuntan el nombre del archivo fuente, el número de página, el título de la sección, la fecha de ingesta, el ID del documento y un hash SHA-256 del contenido original. Estos metadatos habilitan dos capacidades críticas: el filtrado en tiempo de búsqueda (solo documentos de la categoría "contratos" firmados después de 2024-01-01) y la trazabilidad en las respuestas del LLM ("Esta información proviene de la Cláusula 4.2 del Contrato Marco 2024-ESA-0892, página 7").

Finalmente, la etapa de **embedding y persistencia** genera el vector de representación semántica del chunk mediante el modelo de embedding seleccionado y lo escribe en la base vectorial con un ID reproducible basado en el hash del contenido. El uso de IDs basados en hash garantiza la idempotencia del pipeline: si el mismo documento se procesa dos veces (por ejemplo, en un reprocesamiento masivo), el segundo upsert sobrescribe el primero sin crear duplicados ni inconsistencias.

## Fases del ciclo de vida de un documento

- **Extracción:** parsers como PyMuPDF o Unstructured.io convierten el binario en texto plano preservando la estructura semántica; Docling agrega detección de tablas, figuras y layout visual para documentos complejos.
- **Limpieza y normalización:** eliminación de ruido (encabezados y pies de página repetidos, caracteres de control), normalización unicode, corrección de artefactos OCR y detección de idioma para enrutamiento.
- **Chunking:** segmentación en fragmentos de 256–1024 tokens con overlap de 10–20%, balanceando precisión de recuperación y completitud de contexto en cada fragmento.
- **Enriquecimiento de metadatos:** adjuntar a cada chunk información de procedencia estructurada (source, page, section, created\_at, doc\_id) y un hash del contenido para detección de cambios y deduplicación.



- **Embedding y persistencia:** generación del vector de representación semántica y escritura atómica en la base vectorial con ID reproducible; la operación de upsert garantiza idempotencia.
- **Gestión del ciclo de actualización:** detección de cambios mediante comparación de hashes; eliminación del vector obsoleto e inserción del vector actualizado, manteniendo consistencia del índice durante la transición.

**Nota del Arquitecto:** El error más silencioso y devastador del ciclo de vida de un documento es la falta de normalización unicode. Un PDF escaneado que mezcla caracteres unicode de diferentes páginas puede producir secuencias de bytes que el tokenizador del modelo de embedding interpreta como tokens desconocidos, generando embeddings que son vectores de ruido. Un pipeline de limpieza que aplica `unicodedata.normalize('NFC', text)` y filtra caracteres fuera del rango esperado del idioma puede prevenir este tipo de degradación silenciosa que es casi imposible de diagnosticar sin logs detallados por etapa.

El ciclo de vida de un documento es el fundamento sobre el que se construye toda la calidad del sistema RAG. Las secciones siguientes exploran las variantes arquitectónicas del sistema completo (Sección 4) y la decisión de cuándo usar RAG versus alternativas (Sección 5), pero en producción es este pipeline de ingesta el que más frecuentemente es la causa raíz de problemas de calidad.

## Módulo 6 – Capítulo 01 – Sección 04

---

### Tipos de RAG: naive RAG, advanced RAG y modular RAG

El RAG no es una técnica monolítica con una única implementación correcta: es una familia de arquitecturas que evolucionó rápidamente desde su publicación en 2020 a medida que los equipos de ingeniería identificaron los puntos de fallo del enfoque inicial y construyeron soluciones más sofisticadas. Comprender la taxonomía de sistemas RAG —naive, advanced y modular— permite elegir el nivel de complejidad apropiado para cada caso de uso y evitar dos errores opuestos igualmente costosos: subestimar el problema con un naive RAG que no alcanza los umbrales de calidad necesarios, o sobreingeniería con un modular RAG complejo para un caso de uso que un naive RAG resuelve perfectamente bien.

El **naive RAG**, formalizado en el paper original de Lewis et al. (2020), implementa el ciclo básico de embed-retrieve-generate con búsqueda de similitud coseno y prompt de concatenación directa. Su arquitectura es lineal y determinista: para cada query, embedir la query, recuperar los K chunks más similares, concatenarlos en el prompt y llamar al LLM. Implementarlo con LangChain requiere menos de 20 líneas de código: un `VectorStoreRetriever`, un `ChatPromptTemplate` y un `LLMChain`. Sus limitaciones son igualmente simples de describir: Recall@5 típico del 40–55% en benchmarks como BEIR, alta sensibilidad al tamaño del chunk, ausencia de mecanismos para manejar queries léxicamente específicas (nombres propios, códigos de producto), y sin verificación de relevancia entre los chunks recuperados y la query. Estas limitaciones son tolerables para corpus pequeños y homogéneos con queries simples, pero son bloqueantes para sistemas de producción a escala.

El **advanced RAG** aborda estas limitaciones introduciendo mejoras en las tres etapas del pipeline. En pre-retrieval, técnicas como HyDE (Hypothetical Document Embeddings) mejoran el vector de consulta generando un documento hipotético con el LLM antes de la búsqueda, y multi-query retrieval produce múltiples reformulaciones de la query para capturar distintos ángulos semánticos. En retrieval, la búsqueda híbrida combina embeddings densos con BM25 para cubrir tanto la similitud semántica como el matching léxico exacto, y el filtrado de metadatos restringe el espacio de búsqueda a documentos relevantes. En post-retrieval, el reranking con modelos cross-encoder como Cohere Rerank v3 o BGE Reranker

reordena los chunks recuperados por relevancia real, no por similitud coseno aproximada. LlamaIndex y LangChain implementan todos estos patrones como módulos configurables:

`SemanticSplitter`, `MultiQueryRetriever`, `EnsembleRetriever`, `ContextualCompressionRetriever` son los más utilizados.

El **modular RAG**, conceptualizado por Yunfan Gao et al. en 2023, va más allá de las mejoras puntuales para proponer una arquitectura donde el sistema RAG se descompone en módulos independientes e intercambiables: un Search Module que abstrae el mecanismo de recuperación, un Memory Module que gestiona el historial de conversación y el estado entre turnos, un Fusion Module que combina múltiples fuentes de recuperación, y un Routing Module que clasifica cada query y la dirige al pipeline de recuperación apropiado. Este nivel de modularización permite sustituir el retriever vectorial por un retriever basado en grafos de conocimiento sin modificar el pipeline de generación, o añadir una fuente de recuperación web sin cambiar la arquitectura de recuperación sobre el corpus interno. Es la arquitectura natural para sistemas que deben manejar múltiples dominios, múltiples fuentes de conocimiento y múltiples tipos de queries con requerimientos de recuperación distintos.

## Características técnicas de cada variante

- **Naive RAG:** implementación lineal de embed-retrieve-generate sin optimizaciones; Recall@5 típico del 40–55% en benchmarks BEIR; adecuado para corpus menores a 10.000 documentos con queries factualmente simples y homogéneas.
- **Advanced RAG pre-retrieval:** query transformation con HyDE (genera documentos hipotéticos para mejorar el vector de búsqueda) y multi-query (3–5 reformulaciones de la query original con el LLM); mejora Recall@5 en 10–20 puntos sobre naive RAG en corpus con vocabulario técnico especializado.
- **Advanced RAG post-retrieval:** reranking con modelos cross-encoder que re-ordena los chunks recuperados por relevancia semántica real; Cohere Rerank v3 (\$2/millón de unidades) o BGE Reranker v2-m3 (open source, 568M parámetros) mejoran Precision@5 en 15–25 puntos sobre el retriever base.
- **Modular RAG:** desacoplamiento de responsabilidades en módulos independientes; permite routing dinámico donde un clasificador dirige cada query al módulo de recuperación apropiado (vectorial para queries semánticas, BM25 para queries léxicas, SQL para queries sobre datos estructurados).
- **Self-RAG como extensión avanzada:** el LLM emite tokens especiales ([Retrieve], [IsREL], [IsSUP], [IsUSE]) para decidir dinámicamente si recuperar información

adicional durante la generación; requiere fine-tuning del modelo base; cubre el Capítulo 10 en detalle.

- **Routing dinámico:** clasificador (LLM pequeño o modelo de clasificación fine-tuneado) que detecta la categoría de la query (factual, comparativa, temporal, multi-hop) y la dirige al pipeline de recuperación apropiado para esa categoría.

La elección del nivel de complejidad debe estar guiada por métricas de evaluación reales sobre el corpus y las queries objetivo, no por la disponibilidad de frameworks o la complejidad percibida de las técnicas. Un punto de partida sensato es implementar naive RAG, medir Recall@5 y faithfulness sobre el golden dataset, identificar los errores más frecuentes y añadir las mejoras de advanced RAG que específicamente los corrigen. Este enfoque de mejora incremental basada en evidencia evita tanto la sobreingeniería prematura como el estancamiento con un sistema que no alcanza los umbrales de calidad necesarios para los usuarios reales.

La siguiente sección presenta la taxonomía desde otro ángulo: no qué tan sofisticado es el sistema RAG sino cuándo RAG es la arquitectura correcta en primer lugar, comparada con fine-tuning y context stuffing con ventanas de contexto largas.

---

## Módulo 6 – Capítulo 01 – Sección 05

---

### Cuándo usar RAG vs fine-tuning vs contexto largo

Uno de los errores más frecuentes que un AI Engineer comete al inicio de un proyecto es asumir que RAG es siempre la respuesta. La realidad es que el árbol de decisión entre RAG, fine-tuning y context stuffing es una decisión arquitectónica con implicaciones de costo, latencia, mantenibilidad y calidad que dependen del caso de uso específico. Esta sección establece los criterios técnicos y económicos para navegar ese árbol de decisión con evidencia, no con intuición.

La decisión no es binaria entre dos opciones sino entre tres aproximaciones con perfiles de costo-beneficio distintos, y en muchos sistemas de producción la respuesta correcta es una combinación de dos de ellas. Para navegar el árbol con claridad, es útil definir primero las cuatro dimensiones que deben evaluarse para cualquier caso de uso: el **volumen del corpus** (cuántos documentos y cuántos tokens totales), la **frecuencia de actualización** (cada cuánto cambia la información que el sistema necesita), la **necesidad de trazabilidad** (si los usuarios o reguladores necesitan ver la fuente de cada afirmación), y el **presupuesto de inferencia** (el costo y la latencia aceptables por consulta a escala).

El **fine-tuning** (ya sea SFT completo, LoRA, o QLoRA para modelos de parámetros reducidos) es la herramienta correcta cuando el objetivo es adaptar el comportamiento, el estilo de respuesta o el vocabulario especializado del modelo, no inyectarle conocimiento factual actualizable. Un modelo fine-tuneado sobre 10.000 ejemplos de respuestas de atención al cliente en el tono y formato esperado por la empresa aprenderá a generar respuestas con esa estructura; pero si esos ejemplos incluyen precios de productos que cambian mensualmente, los precios aprenderán con overfitting y se degradarán con cada actualización del catálogo. El fine-tuning es una inversión de semanas de trabajo y decenas de miles de dólares en cómputo que solo se justifica cuando el objetivo es el comportamiento del modelo, no su conocimiento. Para la mayoría de los casos de uso empresariales donde el corpus de conocimiento se actualiza con frecuencia, el fine-tuning solo del comportamiento combinado con RAG para el conocimiento factual es la arquitectura correcta.

El **context stuffing** —incluir documentos completos en la ventana de contexto del LLM para responder una consulta— es viable en condiciones muy específicas. Con modelos como Gemini 1.5 Pro (1M tokens de contexto) o Claude 3 Sonnet (200K tokens de contexto con caching), es posible incluir decenas o centenares de documentos medianos en el prompt de una consulta individual. Esto funciona bien para análisis one-off de documentos específicos: "analiza este contrato de 50 páginas y extrae las obligaciones del contratista". Pero el costo por inferencia escala linealmente con los tokens de contexto: una sesión de análisis que incluye 50 documentos de 10 páginas en el contexto puede costar 10–50 veces más que la misma consulta con RAG que recupera solo los 5 chunks relevantes. A escala de millones de consultas, esa diferencia de costo hace al context stuffing operativamente inviable. Adicionalmente, Anthropic publicó investigaciones mostrando que los LLMs atienden de forma desproporcionada al inicio y al final del contexto ("lost-in-the-middle effect"): documentos ubicados en el centro de un contexto muy largo pueden ser ignorados efectivamente, reduciendo la calidad de la respuesta incluso cuando la información relevante está presente.

Un matiz importante para 2025–2026 es el **prompt caching**: Anthropic y Google ofrecen caching de prefijos de prompt a una fracción del costo de los tokens de inferencia estándar. En Claude 3.5 Sonnet, los tokens cacheados cuestan \$0.30/M en lugar de \$3.00/M, una reducción del 90%. Para un corpus de tamaño moderado (100–500 documentos) que raramente cambia, una arquitectura de "RAG estático" donde todo el corpus se incluye en el system prompt cacheado puede ser más simple y económica que implementar un pipeline RAG completo. Este escenario es especialmente relevante para asistentes de dominio muy estrecho con corpus pequeños y estables.

**RAG es la arquitectura correcta** cuando se cumplen uno o más de los siguientes criterios: el corpus supera los 100K tokens totales (punto en el que el context stuffing se vuelve inviable económicamente); el corpus se actualiza con frecuencia diaria o semanal; se requiere trazabilidad de fuentes por regulación o por la naturaleza del servicio (medicina, legal, finanzas); el sistema atiende millones de consultas con un presupuesto de costo por consulta acotado; o la información que el sistema necesita es privada y propietaria, no puede salir de la infraestructura propia.

## Criterios de decisión técnica

- **Fine-tuning es adecuado cuando:** el objetivo es adaptar el comportamiento o estilo del modelo (formato de salida, terminología específica de dominio, tono); el corpus de

comportamiento es relativamente estático y pequeño (menos de 10K ejemplos de entrenamiento).

- **Fine-tuning no resuelve:** la incorporación de conocimiento factual actualizable; los hechos aprendidos con fine-tuning se congelan en la fecha del entrenamiento y se degradan con actualizaciones posteriores del corpus.
- **Context stuffing es viable cuando:** el corpus total cabe en la ventana de contexto del modelo con costo aceptable; la consulta es one-off o de baja frecuencia; el prompt caching de Anthropic/Google hace el costo operativo comparable a RAG para corpus menores de 200K tokens.
- **RAG es la opción correcta cuando:** el corpus supera la ventana de contexto, los documentos se actualizan con frecuencia, se requiere trazabilidad de fuentes, o el sistema atiende millones de consultas con presupuesto acotado.
- **Combinación RAG + fine-tuning:** fine-tuning del LLM para seguir las instrucciones del sistema RAG con mayor fidelidad (responder solo con evidencia del contexto, citar fuentes en el formato esperado), manteniendo RAG como mecanismo de conocimiento; esta combinación es el estado del arte para sistemas enterprise de alta calidad.
- **Métricas de decisión:** calcular costo por consulta para cada alternativa (tokens de contexto  $\times$  precio/token  $\times$  consultas/mes), medir Recall@K del retriever en un dataset de evaluación, medir faithfulness de las respuestas y latencia p95 antes de decidir.

**Nota del Arquitecto:** El 80% de los casos de uso empresariales se resuelven con RAG bien diseñado sin fine-tuning. El consejo práctico es comenzar siempre con RAG naive, construir el golden dataset de evaluación, medir las métricas de calidad, e invertir en fine-tuning solo si el análisis de errores muestra que el problema es el comportamiento del modelo (cómo responde, no qué recupera). Fine-tuning para mejorar un retriever deficiente es dinero desperdiciado.

Esta sección cierra el análisis estratégico del Capítulo 1. La siguiente —y última de este capítulo— posiciona RAG como una disciplina de ingeniería con los mismos estándares de rigor que cualquier sistema distribuido crítico, preparando el terreno conceptual para los nueve capítulos de ingeniería técnica que siguen.

## Módulo 6 – Capítulo 01 – Sección 06

---

### Cierre: RAG como disciplina de ingeniería, no solo integración de APIs

Este capítulo abrió con una pregunta práctica —¿por qué existe RAG?— y terminó con una taxonomía de variantes y un árbol de decisión arquitectónica. Antes de cerrar, es necesario establecer el principio más importante que atraviesa todo el módulo: RAG es una disciplina de ingeniería con la misma complejidad operacional que cualquier sistema distribuido de producción, y tratarlo como una simple integración de APIs es el error que separa los prototipos impresionantes de los sistemas confiables en producción.

La proliferación de frameworks como LangChain, LlamaIndex y Haystack democratizó el acceso a los componentes de RAG: cualquier desarrollador puede construir un sistema que responde preguntas sobre un corpus de documentos en un par de horas. Esta accesibilidad es una fortaleza del ecosistema, pero también generó una falsa impresión: que integrar las APIs es suficiente. En producción, la diferencia entre un prototipo funcional y un sistema confiable radica en las decisiones de ingeniería que los frameworks no toman por el desarrollador. ¿Qué estrategia de chunking es apropiada para los PDFs con layout complejo del corpus? ¿Cómo se detecta que el índice se degradó por documentos actualizados que no fueron reindexados? ¿Cómo se correlacionan trazas entre el retriever y el generador para diagnosticar un aumento de hallucinations? ¿Qué SLO de latencia es adecuado para el caso de uso, y cómo se alerta cuando se viola? Ningún framework responde estas preguntas: las respuestas son responsabilidad del AI Engineer.

Los sistemas RAG de mayor impacto en producción son el resultado de meses de iteración sobre pipelines de evaluación, no de semanas de integración de APIs. El sistema de asistencia de Notion AI, por ejemplo, iteró durante meses sobre la estrategia de chunking específica para las notas de Notion, que tienen una estructura jerarquizada de bloques que los splitters genéricos de LangChain no manejan bien. El equipo de GitHub Copilot invirtió tiempo significativo en el enriquecimiento de metadatos de código (lenguaje, repositorio, archivo, función) que permite filtrar el contexto recuperado al ámbito relevante para el cursor del



usuario. Estos equipos no encontraron las respuestas en la documentación de LangChain: las construyeron mediante experimentación sistemática sobre sus datasets de evaluación.

Tratar RAG como una disciplina de ingeniería implica establecer desde el inicio cuatro prácticas que el resto del módulo desarrolla en detalle: datasets de evaluación golden que midan Recall@K y faithfulness antes de cada release, pipelines de CI/CD para el índice que detecten regresiones automáticamente, SLOs de latencia y calidad con alertas calibradas, y procesos de revisión de degradaciones con análisis de causa raíz. Estas prácticas no son opcionales para sistemas que usuarios reales dependen para tomar decisiones; son el mínimo de rigor ingenieril que distingue un sistema de IA confiable de un sistema que funciona "la mayor parte del tiempo".

## Principio rector

RAG no es una integración sino una arquitectura: cada decisión de diseño en el pipeline de ingesta, indexación, recuperación y generación tiene impacto directo y medible en la calidad de las respuestas que los usuarios experimentan. El framework aporta los bloques de construcción; el ingeniero aporta las decisiones que determinan si el sistema funciona bien.

*"La diferencia entre un programador y un ingeniero de software es que el ingeniero sabe que el sistema seguirá ejecutándose mucho después de que él haya terminado de escribir el código."* — Dave Parnas, pionero de la ingeniería de software modular

El Capítulo 2 entra en el primer componente técnico fundamental del sistema RAG: los modelos de embedding, cómo capturan significado semántico, qué modelos están disponibles en 2025, cómo elegir entre ellos para un dominio específico y cómo evaluarlos con métricas reproducibles. La elección del modelo de embedding es, como veremos, la decisión de mayor impacto sobre el techo de calidad de toda la recuperación.